

**PONTIFÍCIA UNIVERSIDADE CATÓLICA DE CAMPINAS
FACULDADE DE ENGENHARIA DE SOFTWARE**

EDUARDA LUIZA PINHEIRO NEPOMUCENO (24002529)

JEAN YUKI KIMURA (24008214)

JHENIFER LAÍS BARBOSA (24014979)

JOÃO PEDRO DUARTE GIATTI (24019083)

Sched — Documentação Final de Arquitetura

Disciplina: Padrões e Arquitetura de Software

CAMPINAS

2026

Sumário

1	INTRODUÇÃO	1
1.1	Apresentação e justificativa	1
1.2	Objetivos	1
1.3	Problema e público-alvo	1
2	ATRIBUTOS DE QUALIDADE E DECISÕES ARQUITETURAIS	3
2.1	Atributos de qualidade prioritários (ISO/IEC 25010:2023)	3
2.2	Registro de Decisões Arquiteturais (ADRs)	4
3	ESTILO ARQUITETURAL	5
3.1	Plano macro — decomposição em unidades implantáveis	5
3.2	Plano interno — organização do código	5
4	APLICAÇÃO DOS PRINCÍPIOS SOLID	6
4.1	SRP — Single Responsibility Principle	6
4.2	OCP — Open/Closed Principle	6
4.3	LSP — Liskov Substitution Principle	6
4.4	ISP — Interface Segregation Principle	7
4.5	DIP — Dependency Inversion Principle	7
5	CLEAN CODE	8
6	PADRÕES DE PROJETO GOF	9
6.1	Strategy (Comportamental)	9
6.2	Adapter (Estrutural)	10
6.3	Builder (Criacional)	11
7	DESIGN DE API	12
8	CONCLUSÃO	13
	REFERÊNCIAS	14

1 INTRODUÇÃO

1.1 Apresentação e justificativa

O **Sched** é uma plataforma de apoio à decisão para minimercados, padarias e pequenos comércios alimentares. Ele controla produtos, estoque (por lotes com validade), vendas e gera previsões de demanda baseadas em Machine Learning, com o objetivo de proteger o fluxo de caixa e reduzir o desperdício de alimentos. O backend já existia (originário do Projeto Integrador V) e foi aqui evoluído e organizado para evidenciar, de forma demonstrável, uma arquitetura justificada, baseada nos princípios SOLID, Clean Code e padrões GoF.

O escopo desta refatoração concentrou-se exclusivamente na pasta **api/** (backend desenvolvido em Spring Boot). O frontend, a pasta **ai/** (serviço de IA em Flask) e o banco de dados relacional foram totalmente preservados sem alterações estruturais.

1.2 Objetivos

- **Geral:** Prover uma API segura, extensível e manutenível para a gestão de estoque e vendas, contendo previsão de demanda integrada e isolamento lógico por empresa.
- **Específicos:**
 - Registrar produtos e lotes de estoque associados a datas de validade rígidas;
 - Registrar vendas com baixa automática de estoque baseada no algoritmo FIFO (*First-In, First-Out*), garantindo que lotes com vencimento mais próximo saiam primeiro;
 - Emitir alertas automatizados de vencimento próximo e de estoque baixo;
 - Integrar de forma desacoplada a um serviço externo de Inteligência Artificial para previsão de demanda e recomendação de reposição;
 - Garantir o isolamento estrito *multi-tenant*, onde cada empresa visualiza unicamente os seus próprios dados corporativos através de autenticação via tokens JWT.

1.3 Problema e público-alvo

Pequenos comércios alimentares operam historicamente com margens de lucro apertadas e um alto risco latente de perdas financeiras por vencimento de produtos. Atualmente, faltam ferramentas acessíveis no mercado que cruzem de forma simples os dados de estoque corrente, histórico de vendas e previsão preditiva de demanda.

O público-alvo deste sistema compreende proprietários e operadores de minimercados ou padarias que necessitam tomar decisões rápidas de compra e reposição. As regras de

negócio não são triviais, pois envolvem o gerenciamento de filas FIFO para lotes, isolamento *multi-tenancy* em nível de aplicação, segurança de dados comerciais, regras de alertas extensíveis e integração de rede com serviços externos distribuídos.

2 ATRIBUTOS DE QUALIDADE E DECISÕES ARQUITETURAIS

2.1 Atributos de qualidade prioritários (ISO/IEC 25010:2023)

1. Manutenibilidade (Modularidade, Testabilidade, Reusabilidade):

- *Justificativa:* Tratando-se de uma equipe de desenvolvimento pequena e de um sistema sob evolução contínua, o custo de engenharia dominante baseia-se na capacidade de alterar o código com segurança.
- *Decisões arquiteturais:* Adoção de uma arquitetura em camadas combinada a um monolito modular (ADR-001); aplicação do padrão *Strategy* para o motor de alertas (ADR-004); implementação do padrão *Port/Adapter* para isolar o serviço de IA (ADR-005); e a substituição completa de utilitários estáticos pelo componente injetável `AuthenticatedUserProvider` (ADR-007).
- *Métricas observáveis:* Número de classes alteradas para a adição de um novo modelo de alerta (meta: 1 classe, atingida pelo uso do *Strategy*); cobertura de testes unitários concentrada nas classes de serviço (6 classes de teste estruturadas); monitoramento do acoplamento aferente e eferente por pacotes.

2. Segurança (Confidencialidade, Integridade, Autenticidade):

- *Justificativa:* O sistema armazena dados comerciais e financeiros altamente sensíveis de múltiplas empresas concorrentes em uma mesma base de dados, tornando o vazamento de informações entre *tenants* inaceitável.
- *Decisões arquiteturais:* Utilização de arquitetura de segurança baseada em JWT *stateless* e criptografia BCrypt (ADR-002); aplicação de isolamento mandatório por `company_id` em todas as consultas SQL e validações de serviço (ADR-003); remoção total do segredo padrão `insecure` da aplicação e fechamento de endpoints expostos da IA.
- *Métricas observáveis:* Número de endpoints expostos publicamente (restritos apenas ao `login`, criação de `company` e à especificação aberta); ausência absoluta de segredos textuais em código fonte (*hardcoded*); testes automatizados específicos contra acesso cruzado de dados entre empresas distintas.

3. Confiabilidade (Maturidade, Tolerância a Falhas, Integridade de Dados):

- *Justificativa:* Falhas na baixa de estoques ou inconsistências de vendas corrompem diretamente o balanço financeiro e o planejamento logístico do comerciante.
- *Decisões arquiteturais:* Demarcação de todas as operações de escrita com a anotação `@Transactional` (garantindo atomicidade entre a baixa FIFO de múltiplos lotes e o registro da venda); implementação de exclusão lógica via

soft delete para preservação de histórico analítico (ADR-006); validação rígida de saldo suficiente antes da persistência; tratamento centralizado através de um `GlobalExceptionHandler`; e degradação suave do adaptador de IA (nunca retornando `null`, mas sim coleções vazias).

- *Métricas observáveis*: Taxa de erros do tipo 5xx monitorados via Spring Boot Actuator; ausência histórica de registros com estoque negativo; consistência transacional validada por testes de estresse.

2.2 Registro de Decisões Arquiteturais (ADRs)

As decisões arquiteturais críticas foram tomadas e padronizadas no formato canônico através de registros estruturados, resumidos na Tabela 1.

Tabela 1: Resumo dos Registros de Decisões Arquiteturais (ADRs).

ADR	Título
001	Monolito modular organizado em camadas
002	Autenticação stateless via JWT
003	Isolamento multi-tenant por <code>company_id</code>
004	Strategy + Registry para regras de alerta
005	Port/Adapter para o serviço de IA
006	Exclusão lógica (soft delete)
007	Provedor de usuário autenticado injetável (DIP)
008	Especificação OpenAPI como contrato de API

3 ESTILO ARQUITETURAL

3.1 Plano macro — decomposição em unidades implantáveis

O sistema adota o estilo de **monolito modular**, sendo compilado em um único artefato Spring Boot executável. Essa abordagem é justificada na ADR-001: as dimensões reduzidas da equipe, o domínio altamente coeso do sistema e a ausência inicial de requisitos de escalabilidade independente eliminam a necessidade de adoção de microsserviços, que adicionariam custos operacionais sem retorno financeiro imediato. A única fronteira distribuída da aplicação consiste no serviço externo de IA (Flask), o qual é consumido por meio do padrão *Port/Adapter* acoplado a protocolos HTTP/JSON, isolando o núcleo do negócio de mudanças de provedores tecnológicos.

3.2 Plano interno — organização do código

A estrutura interna baseia-se em uma **arquitetura em camadas** tradicional com forte isolamento e contornos hexagonais em suas fronteiras de comunicação externa:

- **controller:** Responsável pela adaptação do protocolo HTTP de entrada (`AuthController`, `ProductController`).
- **service:** Concentrador exclusivo das regras de negócio do domínio (`ProductService`, `SaleService`, `AlertService`).
- **repository:** Camada de acesso e persistência de dados gerenciada pelo Spring Data JPA (`ProductRepository`, `StockRepository`).
- **domain:** Modelagem de entidades ricas do sistema (`Company`, `User`, `Product`, `Stock`, `Sale`).
- **dto:** Estruturas de dados imutáveis (*records*) para contratos de entrada e saída.
- **security:** Mecanismos de criptografia, filtros de requisição de rede JWT e contexto (`SecurityConfig`, `AuthenticatedUserProvider`).
- **client:** Implementação da porta e adaptadores do ecossistema de Inteligência Artificial (`DemandForecastPort`, `FlaskDemandForecastClient`).
- **exception:** Centralização de erros e manipuladores globais (`GlobalExceptionHandler`).

4 APLICAÇÃO DOS PRINCÍPIOS SOLID

4.1 SRP — Single Responsibility Principle

Cada classe possui apenas uma única razão para ser alterada. O componente `AlertService`, por exemplo, atua exclusivamente selecionando e delegando tarefas; a responsabilidade de saber *como* processar e calcular cada alerta específico pertence unicamente às regras correspondentes injetadas.

```
@Service
public class AlertService {
    private final Map<AlertType, StockAlertRule> rulesByType;
    private final AuthenticatedUserProvider authenticatedUserProvider;

    private List<AlertResponse> evaluate(AlertType type) {
        User authUser = authenticatedUserProvider.getCurrentUser();
        Long companyId = authUser.getCompany().getId();
        return rulesByType.get(type).evaluate(companyId);
    }
}
```

4.2 OCP — Open/Closed Principle

A inclusão de novos tipos ou formatos de alertas no sistema **não exige nenhuma modificação** interna no código-fonte do componente `AlertService`. O sistema encontra-se aberto para extensão e fechado para modificação.

```
public interface StockAlertRule {
    AlertType type();
    List<AlertResponse> evaluate(Long companyId);
}

@Component
public class LowStockRule implements StockAlertRule {
    public AlertType type() { return AlertType.LOW_STOCK; }
    public List<AlertResponse> evaluate(Long companyId) { /* ... */ }
}
```

4.3 LSP — Liskov Substitution Principle

Qualquer classe que implemente a interface `DemandForecastPort` pode substituir o adaptador de produção sem causar quebras estruturais ou comportamentais indesejadas

ao `AIService`, dado que o contrato estabelecido assegura que coleções nulas nunca serão retornadas (degradando estritamente para listas vazias).

```
@Override
public List<AIPredictionResponse> forecast(List<AIDemandDataRequest>
    demandData) {
    AIPredictionResponse[] response = restTemplate.postForObject(forecastUrl,
        demandData, AIPredictionResponse[].class);
    return response == null ? List.of() : List.of(response);
}
```

4.4 ISP — Interface Segregation Principle

As interfaces expostas pelo sistema são altamente específicas e enxutas. A interface `StockAlertRule` declara unicamente os métodos `type()` e `evaluate()`, enquanto `DemandForecastPort` declara estritamente o método `forecast()`. Nenhum cliente do sistema é induzido a depender de métodos abstratos que não utiliza, simplificando a criação de dublês de testes (*mocks*).

4.5 DIP — Dependency Inversion Principle

Os módulos de alto nível não se acoplam a detalhes de baixo nível, mas dependem estritamente de abstrações. O `AIService` consome a porta aberta de previsão de demanda em vez do cliente HTTP direto (`RestTemplate`). Complementarmente, os serviços de negócio agora dependem da interface injetável `AuthenticatedUserProvider`, eliminando o acoplamento estático anterior ao contexto global do Spring Security.

5 CLEAN CODE

Diversas práticas de legibilidade e engenharia de software limpa foram introduzidas durante a refatoração do código-fonte do sistema:

- **Nomes claros e expressivos:** O método `getProductStockSummary()` substituiu o antigo termo genérico e semanticamente incorreto `findById()` exposto no `StockController`. Adicionalmente, a constante expressiva `Stock.NO_EXPIRATION` substituiu o uso disseminado de números mágicos temporais (ex: `LocalDateTime.of(1970,1,1,0)`).
- **Funções pequenas e coesas:** Métodos de serviços foram refatorados para manter baixa complexidade ciclomática, extraíndo conversões de dados para subfunções isoladas de mapeamento como `mapToResponse()` e `mapToBatchResponse()`.
- **Remoção de duplicidade:** Exclusão de consultas SQL redundantes em base de dados, como chamadas repetidas a `userRepository.findByEmail()` no interior do componente `SaleService`, visto que o principal já trafegava a entidade de usuário autenticada.
- **Tratamento explícito de erros:** Exceções de negócio customizadas (`InsufficientStockException`, `ProductHasStockException`, `ResourceNotFoundException`) passaram a ser interceptadas de forma centralizada pelo `GlobalExceptionHandler`, que agora formata respostas HTTP estruturadas devolvendo erros do tipo 400 (*Bad Request*) mapeando chaves e mensagens detalhadas de validações falhas do componente `@Valid`.
- **Testabilidade:** Criação de 6 classes de testes unitários utilizando JUnit 5 e Mockito, cobrindo de maneira isolada regras críticas como a ordem de escoamento FIFO de lotes, validações *multi-tenant*, lógica dos alertas de estoque e operações de CRUD padrão.

6 PADRÕES DE PROJETO GOF

6.1 Strategy (Comportamental)

- **Problema:** Evitar que a introdução de novas lógicas de alerta gere estruturas condicionais complexas (`if-else` ou `switch`), violando o princípio OCP.
- **Solução:** Criação de uma interface comum (*Strategy*) implementada de forma independente por cada regra de negócio, gerenciada por um mapa indexador (*Registry*).

```
// ESTRATEGIA: Interface comum para as regras de alerta
public interface StockAlertRule {
    AlertType type();
    List<AlertResponse> evaluate(Long companyId);
}

// CONCRETIZACAO 1: Regra para Produtos Vencendo
@Component
public class ExpiringStockRule implements StockAlertRule {
    @Override
    public AlertType type() { return AlertType.EXPIRING; }

    @Override
    public List<AlertResponse> evaluate(Long companyId) { // ...retorna os
        produtos proximos de vencer}
}

// CONCRETIZACAO 2: Regra para Estoque Baixo
@Component
public class LowStockRule implements StockAlertRule {
    @Override
    public AlertType type() { return AlertType.LOW_STOCK; }

    @Override
    public List<AlertResponse> evaluate(Long companyId) { // ...retorna os
        produtos com baixo estoque}
}

// CONTEXTO: Centraliza e delega a execucao dinamicamente atravs do Registry
@Service
public class AlertService {
    private final Map<AlertType, StockAlertRule> rulesByType;

    public AlertService(List<StockAlertRule> rules) {
        this.rulesByType =
```

```

        rules.stream().collect(Collectors.toMap(StockAlertRule::type,
        Function.identity()));
    }

    private List<AlertResponse> evaluate(AlertType type, Long companyId) {
        return rulesByType.get(type).evaluate(companyId);
    }
}

```

- **Benefício:** O `AlertService` torna-se totalmente extensível. A inclusão de um novo tipo de alerta exige apenas a criação de uma nova classe que implemente `StockAlertRule`, sem necessidade de modificar nenhuma linha de código do serviço central.

6.2 Adapter (Estrutural)

- **Problema:** Evitar que o núcleo de negócios e o serviço de domínio da aplicação fossem expostos ou rigidamente amarrados às particularidades de infraestrutura, tais como o protocolo de comunicação HTTP, uso do `RestTemplate`, strings de URLs e chaves JSON do microserviço externo em Flask.
- **Solução:** Aplicação do padrão *Adapter* em conformidade com as diretrizes de uma arquitetura hexagonal. Foi estabelecida uma interface abstrata de domínio que atua como uma porta de saída (*Port*). Em seguida, implementou-se um componente de infraestrutura (*Adapter*) responsável por interceptar as regras de negócio, realizar o mapeamento físico da requisição de rede HTTP via `RestTemplate` e consumir a Inteligência Artificial externa de forma totalmente transparente para o núcleo do sistema, conforme ilustrado abaixo:

```

// A PORTA: Abstracao de dominio independente de infraestrutura
public interface DemandForecastPort {
    List<AIPredictionResponse> forecast(List<AIDemandDataRequest> demandData);
}

// O ADAPTADOR: Encapsula as chamadas de rede HTTP e o RestTemplate
public class FlaskDemandForecastClient implements DemandForecastPort {
    @Override
    public List<AIPredictionResponse> forecast(List<AIDemandDataRequest>
        demandData) {
        ...
    }
}

```

```
// O CLIENTE DO PADRAO: Servico de Dominio acoplado apenas a Porta
public class AIService {
    private final DemandForecastPort demandForecastPort;

    public List<AIPredictionResponse> getPredictions() {
        return demandForecastPort.forecast(getDemandData());
    }
}
```

- **Benefícios:** Atua como uma robusta camada anticorrupção (*Anticorruption Layer*). Caso o microsserviço em Flask seja substituído por um provedor em nuvem, por filas de mensageria ou por outra biblioteca de aprendizado profundo, o componente `AIService` permanecerá intacto. Além disso, a testabilidade é drasticamente otimizada, permitindo simular falhas e comportamentos da inteligência artificial por meio do isolamento de mocks da interface de modo trivial.

6.3 Builder (Criacional)

- **Problema:** A instanciação manual de entidades de domínio complexas contendo múltiplos atributos tornava os construtores ilegíveis e propensos a falhas de posicionamento de parâmetros.
- **Solução:** Acoplamento da anotação `@Builder` (Lombok) sobre as classes de entidade.
- **Benefícios:** Escrita fluente, legível e garantia de imutabilidade parcial durante as etapas de construção de dados nos serviços.
- **Exemplo de uso:**

```
Stock stock = Stock.builder()
    .quantity(dto.quantity())
    .expirationDate(dto.expirationDate())
    .product(product)
    .createdBy(authUser)
    .build();
```

7 DESIGN DE API

A API do sistema Sched foi projetada seguindo o modelo de maturidade REST sobre o protocolo HTTP, trafegando estruturas no formato JSON e estabelecendo controle de acessos via cabeçalhos de autenticação **Bearer JWT**.

A especificação contratual completa da aplicação encontra-se mapeada no arquivo `docs/openapi.yaml` sob o padrão OpenAPI 3.0.3, descrevendo de forma explícita rotas, parâmetros de URL, esquemas de requisição/resposta, barramentos de segurança e dicionários de tratamento de falhas.

O versionamento adota a abordagem explícita inserida diretamente no caminho de navegação (`/api/v1/...`) para gerenciar quebras de compatibilidade futuras. As coleções de dados retornadas realizam a listagem completa por empresa devido ao volume controlado de dados de pequenos comércios; contudo, a paginação de registros baseada na interface **Pageable** do Spring Data já se encontra padronizada no documento de arquitetura como evolução obrigatória para os endpoints de maior tráfego volumétrico (`/stock` e `/sale`).

As falhas estruturais de validações geram respostas padronizadas sob o formato **ErrorDetails**, encapsulando os campos `timestamp`, `status`, `error` e `message`. Em cenários de violação de restrições de validação (`@Valid`), a API retorna o status HTTP 400 acompanhada por um mapa chave-valor detalhando o campo causador e a respectiva mensagem de erro de validação.

A distribuição macro de caminhos da API compreende: autenticação (`/auth/*`), gestão de usuários (`/user/*`), parametrização de empresas (`/company/*`), cadastro de produtos (`/product/*`), controle de estoque (`/stock/*`), registro de vendas (`/sale/*`), motores de alertas (`/alert/*`) e previsões preditivas (`/ai/predictions`).

8 CONCLUSÃO

A refatoração incremental do backend do projeto Sched validou as premissas estabelecidas, preservando as regras de negócio legadas enquanto elevava a qualidade arquitetural do ecossistema de software. A explicitação de padrões consolidados de engenharia (como *Strategy*, *Adapter* e a inversão de dependências via portas de isolamento) proporcionou maior robustez contra alterações de código.

Vulnerabilidades de segurança críticas foram sanadas através da eliminação de chaves criptográficas em texto claro e fechamento de endpoints vulneráveis da inteligência artificial. A desacoplamento de chamadas estáticas do Spring Security aumentou drasticamente o índice de testabilidade unitária da aplicação.

Como limitações da iteração atual, destaca-se a ausência de mecanismos automáticos para invalidação de tokens JWT antes da janela de expiração (*blocklisting*), falta de paginação nativa nos endpoints de listagem de estoque e dependência de atualizações manuais no arquivo de contrato OpenAPI. Estas dívidas técnicas foram catalogadas e priorizadas como melhorias em ciclos futuros de engenharia, recomendando-se a introdução da biblioteca `springdoc-openapi` para geração automatizada de documentação e desenvolvimento de suítes de testes de integração *end-to-end*.

REFERÊNCIAS

FOWLER, M. **Patterns of Enterprise Application Architecture**. Boston: Addison-Wesley, 2002.

GAMMA, E.; HELM, R.; JOHNSON, R.; VLISSIDES, J. **Design Patterns: Elements of Reusable Object-Oriented Software**. Boston: Addison-Wesley, 1994.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 25010:2023** — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model. Genebra: ISO, 2023.

MARTIN, R. C. **Clean Code: A Handbook of Agile Software Craftsmanship**. Upper Saddle River: Prentice Hall, 2008.

MARTIN, R. C. **Clean Architecture: A Craftsman's Guide to Software Structure and Design**. Boston: Prentice Hall, 2017.

NYGARD, M. **Documenting Architecture Decisions**. 2011. Disponível em: <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>. Acesso em: 5 jun. 2026.